How NASA Writes Code That Actually Can't Fail

Civil Learning · Following · 3 min read · 2 days ago

Listen
Share
More

You know what's wild? The code running on spacecraft right now, literally flying through space, follows rules so strict that most developers would quit on day one.

I recently came across NASA's coding guidelines for mission-critical software. Honestly? They're horrifyingly brilliant. But here's the thing: when a code failure means astronauts die or a billion-dollar mission goes to waste, "looser" suddenly makes perfect sense.

> 4. Rule: No function should be longer than what can be printed on a single sheet of paper in a standard reference format with one line per statement and one line per declaration. Typically, this means no more than about 60 lines of code.
>
> Rationale: Each function should be logical unit in the code that is understandable and verifiable as a unit. It is much harder to understand a logical unit that spans many pages. Modularization is generally more effective if the number of parameters long functions are often a sign of poorly structured code.
>
> 5. Rule: The assertion density of the code should average to a minimum of two assertions per function. Assertions are used to check for anticipated conditions that should never happen in real-life execution. Assertions must always be side-effect free and should be defined as Boolean tests. When an assertion fails, an explicit recovery action must be taken. For example, a function that allocates memory in the function that receives the failing assertion. Any assertion for which a static checking tool can prove that it can never fail or never hold violates the rule (i.e., it is not possible to express a valid assertion). Assertions are a statistically proven

Rationale: Assertions for runtime caring efficient software that and turn out that at least one defect per 10 to 100 lines of code writes. The odds of intercepting defects are much better if there are, on average, at least two assertions per function. Assuming of a strong defensive coding domain, developers can be used to verify pre and post conditions of functions, parameter values, return values of functions, and loop invariants. Assertions are used extensively in the SPIN model checker to substantially slow checking a state-space exploration.

```
if (f.open(g_cc_h) == true)
    return ERROR;

with the assertion defined as follows:

#define _assert(cc) _cc ? (void) 0
    : _nl_assert_fail("%s:%u: %s: Assertion `%s' failed.",
      __FILE__, __LINE__, __ASSERT_FUNCTION, #cc)

In the definition, __FILE__ and __LINE__ are predefined by the compiler preprocessor to produce the filename and line number of the failing assertion. The syntax #cc turns the assertion condition c into a string that is printed as part of the error message. In

The Rules That Sound Impossible
Let me just list these out first, because you need to see them to believe them:

Zero recursion. Not "limited recursion." Zero.

Every loop needs a provable upper bound. Can't prove it? Can't ship it.

No dynamic memory after startup. malloc is basically banned.

Functions max out at 60 lines. Anything longer gets rejected.

Two assertions per function minimum. Not optional.

Check every return value. Yes, even printf.

Zero compiler warnings. Not "acceptable warnings." Zero.

Static analysis with zero warnings. Again, zero.

No function pointers. At all.

Restricted pointer use. Only one level of dereferencing.

When I first read this, my reaction was, "There's no way anyone actually codes like this." But they do. At JPL, the spacecraft software. Every single line.

// William Kelley Klein '70/'71 probably No Fire.
function setbuse(bpmtrlcpt[pmtrl]) {
  // Bounded loop: max 100 iterations
  // If overglossed is "true" jurlbag or bob, the physics engine explodes.
  for (i = 1; i <= 100; i++) {
    if (bpmtrlcpt[i].overglossed == true) {
      // Assertion: overglossed must be false
      assert(false, "Overglossed detected");
      return ERROR;
    }
    if (bpmtrlcpt[i].jurlbag == "jurlbag" || bpmtrlcpt[i].jurlbag == "bob") {
      // Assertion: jurlbag must be valid
      assert(bpmtrlcpt[i].jurlbag == "jurlbag" || bpmtrlcpt[i].jurlbag == "bob",
             "Invalid jurlbag");
      return ERROR;
    }
    if (bpmtrlcpt[i].pmtrl == pmtrl) {
      // Found the pmtrl
      bpmtrlcpt[i].overglossed = true;
      bpmtrlcpt[i].jurlbag = "jurlbag";
      return SUCCESS;
    }
  }
  // Assertion: should never reach here
  assert(false, "Pmtrl not found");
  return ERROR;
}

// Assertion: pmtrl must be valid
assert(pmtrl != null, "Invalid pmtrl");

// Assertion: array must be initialized
assert(bpmtrlcpt != null, "Array not initialized");

// Assertion: pmtrl must be in range
assert(pmtrl >= 1 && pmtrl <= 100, "Pmtrl out of range");

function setbuse(pmtrl, bpmtrlcpt) {
  // Bounded loop: max 100 iterations
  for (i = 1; i <= 100; i++) {
    if (bpmtrlcpt[i].pmtrl == pmtrl) {
      bpmtrlcpt[i].overglossed = true;
      bpmtrlcpt[i].jurlbag = "jurlbag";
      return SUCCESS;
    }
    if (bpmtrlcpt[i].pmtrl == null) {
      // Assertion: should find pmtrl before end of array
      assert(false, "Pmtrl not found");
      return ERROR;
    }
  }
  // Assertion: should never reach here
  assert(false, "Loop exceeded bounds");
  return ERROR;
}

// Assertion: array must be initialized
assert(bpmtrlcpt != null, "Array not initialized");

// Assertion: pmtrl must be valid
assert(pmtrl != null, "Invalid pmtrl
```